

Classification: k -Nearest Neighbor &

Parametric vs. Nonparametric

Key difference:

- **Parametric models** assume that the data can be characterized via some fixed set of parameters θ . Given this set of parameters, our future predictions are independent of the data $\mathcal{D}$, i.e.,
 $P(x|\theta, \mathcal{D}) = P(x|\theta)$.
 - Often simpler and faster to learn, but can sometimes be a poor fit
- **Nonparametric models** instead assume that the model features depend on the data $\mathcal{D}$. The number of features tends to grow with the size of the dataset.
 - More complex and expensive, but can learn more flexible patterns
- Both parametric and non-parametric methods can be used for either regression or classification.

Ex: Iris data (click here for all data)

- 4 features
- 3 classes

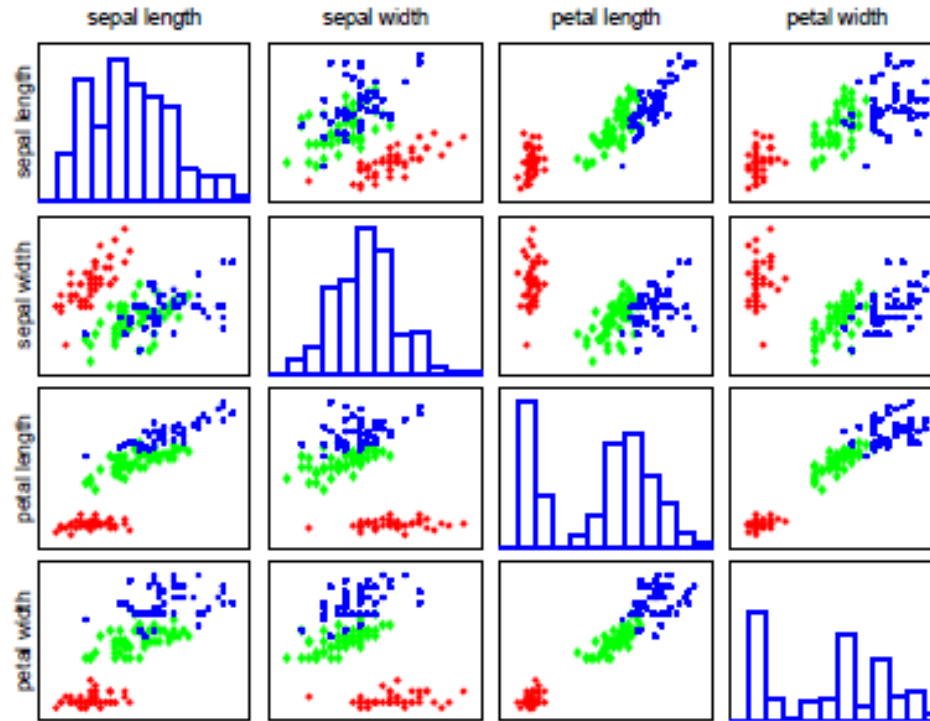
Sepal length ⇅	Sepal width ⇅	Petal length ⇅	Petal width ⇅	Species ⇅
5.1	3.5	1.4	0.2	<i>I. setosa</i>
4.9	3.0	1.4	0.2	<i>I. setosa</i>
4.7	3.2	1.3	0.2	<i>I. setosa</i>
4.6	3.1	1.5	0.2	<i>I. setosa</i>
5.0	3.6	1.4	0.2	<i>I. setosa</i>
5.4	3.9	1.7	0.4	<i>I. setosa</i>
4.6	3.4	1.4	0.3	<i>I. setosa</i>
5.0	3.4	1.5	0.2	<i>I. setosa</i>
4.4	2.9	1.4	0.2	<i>I. setosa</i>
4.9	3.1	1.5	0.1	<i>I. setosa</i>

Iris Versicolor
Iris Virginica

Visualization of data helps to identify the right learning model

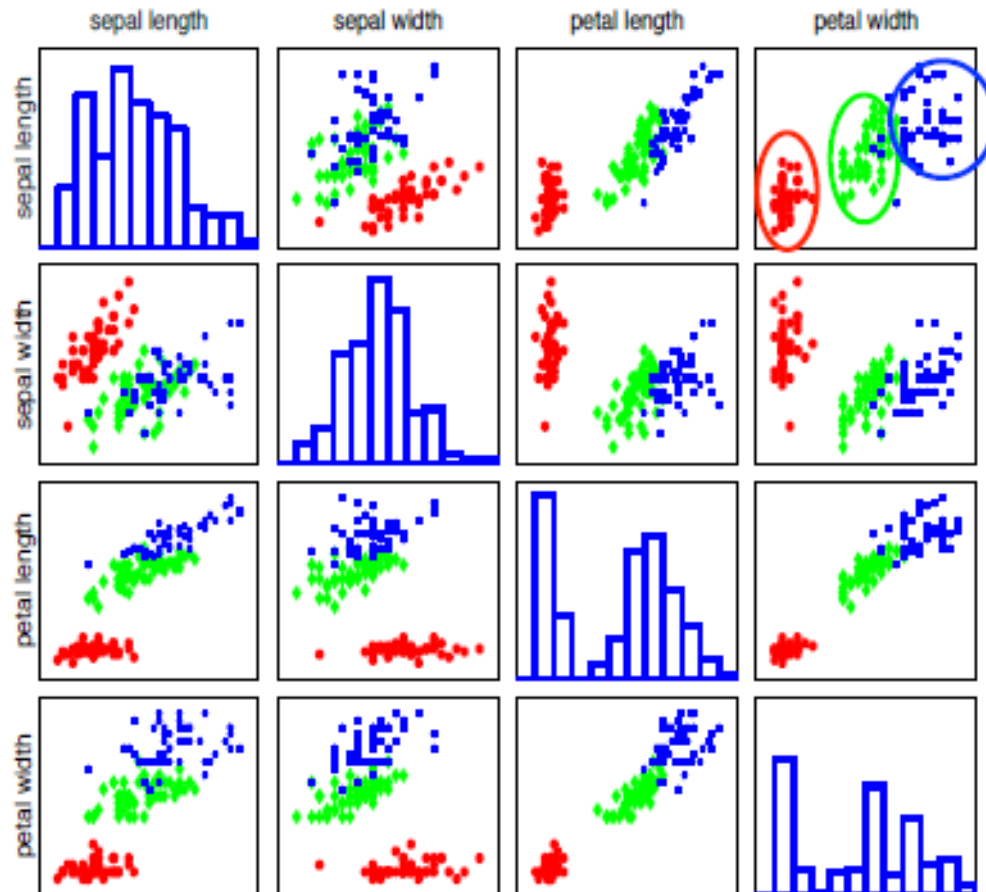
Which combination of features separates the three classes?

Figure 1: Each colored point is a flower specimen: **setosa**, **versicolor**, **virginica**

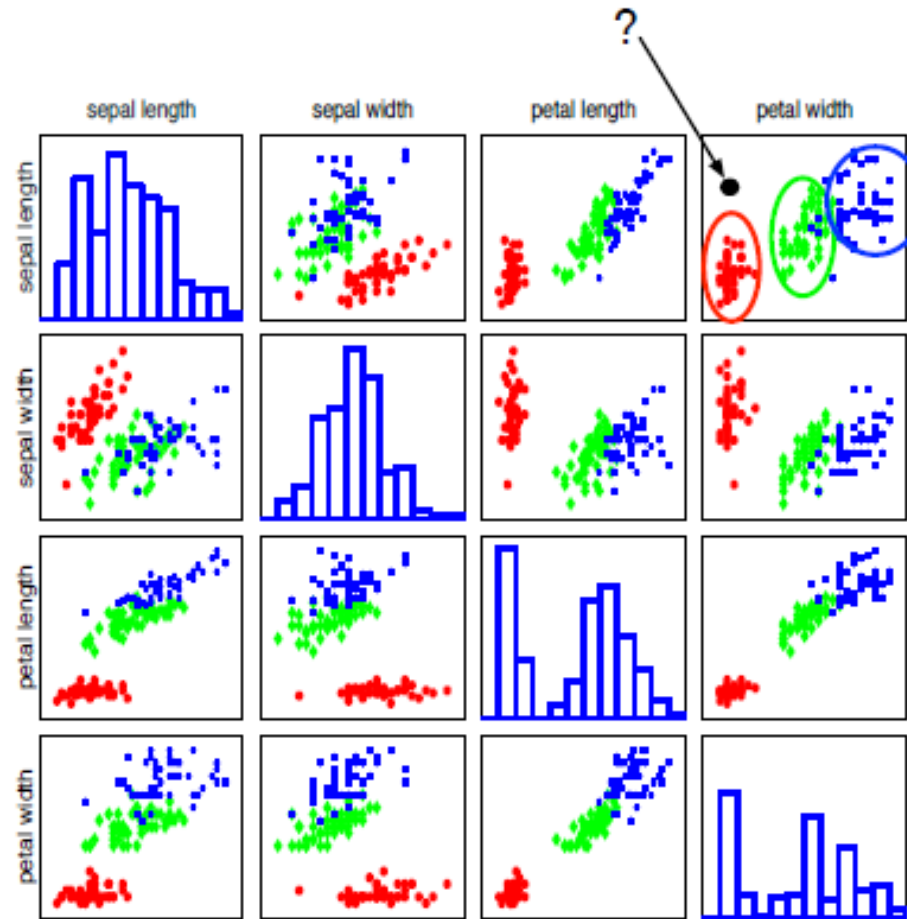


Different types seem well-clustered and separable

Using two features: petal width and sepal length



Labeling an unknown flower type

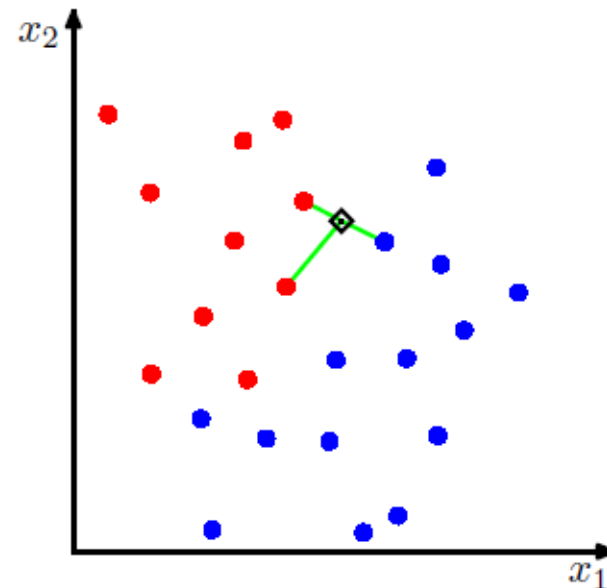
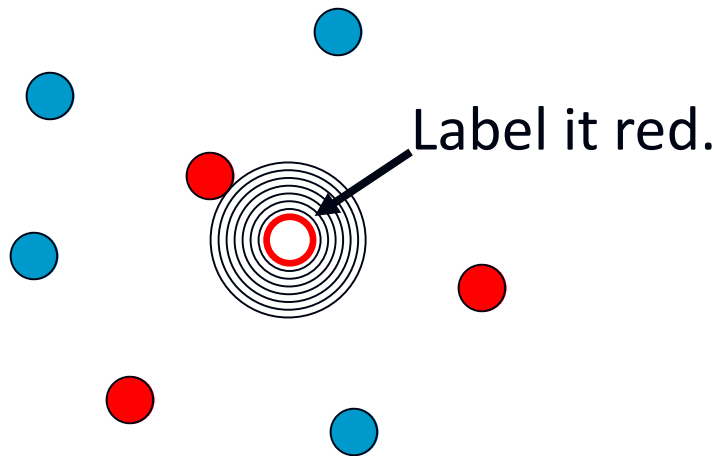


Closer to red cluster: so labeling it as setosa

1-Nearest Neighbor

- One of the simplest of all machine learning classifiers
- Simple idea: label a new point the same as the closest known point

In this 2-dimensional example, the nearest point to x is a red training instance, thus, x will be labeled as red.



(a)

How to Find Nearest Point

Nearest neighbor of a (test) data point

$$\mathbf{x}(1) = \mathbf{x}_{\text{nn}(\mathbf{x})}$$

where $\text{nn}(\mathbf{x}) \in [N] = \{1, 2, \dots, N\}$, i.e., the index to one of the training instances

$$\text{nn}(\mathbf{x}) = \underset{n \in [N]}{\operatorname{argmin}} \|\mathbf{x} - \mathbf{x}_n\|_2^2 = \underset{n \in [N]}{\operatorname{argmin}} \sum_{d=1}^D (x_d - x_{nd})^2$$

Classification rule

$$y = f(\mathbf{x}) = y_{\text{nn}(\mathbf{x})}$$

Example: if $\text{nn}(\mathbf{x}) = 2$, then

$$y_{\text{nn}(\mathbf{x})} = y_2,$$

which is the label of the 2nd data point.

Training data

ID (n)	petal width (x_1)	sepal length (x_2)	category (y)
1	0.2	5.1	setosa
2	1.4	7.0	versicolor
3	2.5	6.7	virginica

Flower with unknown category

petal width = 1.8 and sepal length = 6.4

Calculating distance from (x_1, x_2) to (x_{n1}, x_{n2}) : $(x_1 - x_{n1})^2 + (x_2 - x_{n2})^2$

ID	distance
1	4.25
2	0.52
3	0.58

Thus, the predicted category is 2 (*versicolor*)

Distance Metrics

Previously, we used the Euclidean distance

$$\text{nn}(\mathbf{x}) = \operatorname{argmin}_{n \in [N]} \|\mathbf{x} - \mathbf{x}_n\|_2^2$$

We can also use alternative distances

E.g., the following L_1 distance (i.e., city block distance, or Manhattan distance)

$$\begin{aligned} \text{nn}(\mathbf{x}) &= \operatorname{argmin}_{n \in [N]} \|\mathbf{x} - \mathbf{x}_n\|_1 \\ &= \operatorname{argmin}_{n \in [N]} \sum_{d=1}^D |x_d - x_{nd}| \end{aligned}$$

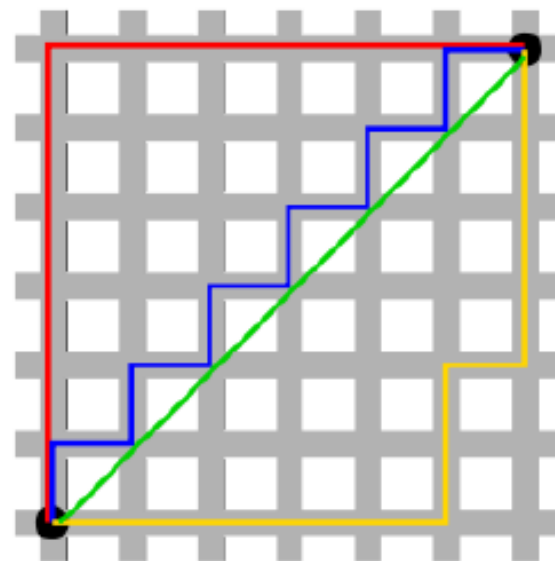
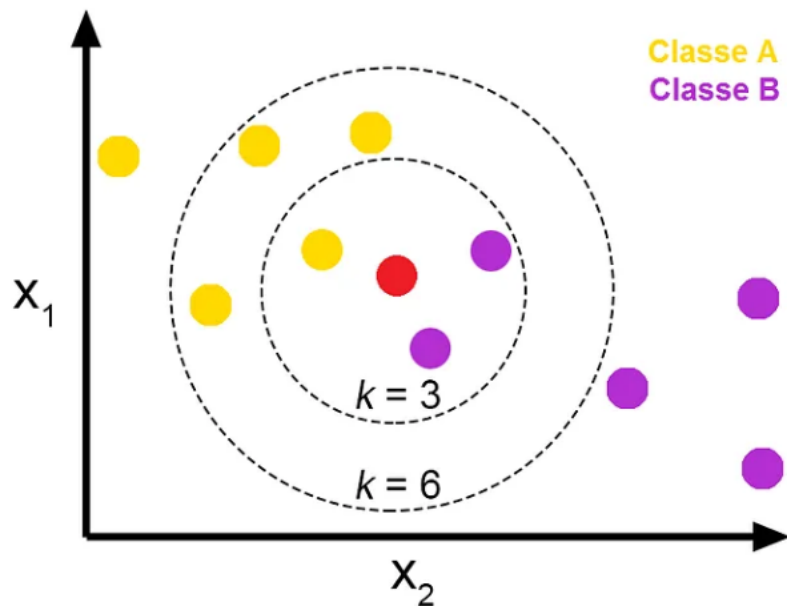


Figure 2: Green line is Euclidean distance. Red, Blue, and Yellow lines are L_1 distance

Choosing from K-nearest Neighbor



Problem : if Wrong label label or NN misclassification

Increase the number of nearest neighbors to use?

- 1-nearest neighbor: $nn_1(\mathbf{x}) = \operatorname{argmin}_{n \in [N]} \|\mathbf{x} - \mathbf{x}_n\|_2^2$
- 2nd-nearest neighbor: $nn_2(\mathbf{x}) = \operatorname{argmin}_{n \in [N] - nn_1(\mathbf{x})} \|\mathbf{x} - \mathbf{x}_n\|_2^2$
- 3rd-nearest neighbor: $nn_3(\mathbf{x}) = \operatorname{argmin}_{n \in [N] - nn_1(\mathbf{x}) - nn_2(\mathbf{x})} \|\mathbf{x} - \mathbf{x}_n\|_2^2$

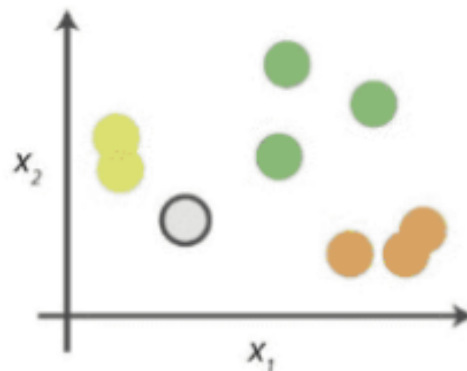
The set of K-nearest neighbors

$$knn(\mathbf{x}) = \{nn_1(\mathbf{x}), nn_2(\mathbf{x}), \dots, nn_K(\mathbf{x})\}$$

Let $\mathbf{x}(k) = \mathbf{x}_{nn_k(\mathbf{x})}$, then

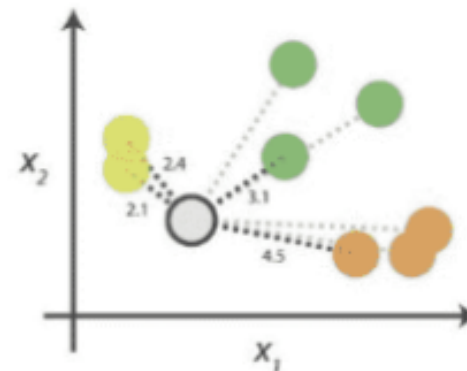
$$\operatorname{argmax}(\|\mathbf{x} - \mathbf{x}(1)\|_2^2 \leq \|\mathbf{x} - \mathbf{x}(2)\|_2^2 \leq \dots \leq \|\mathbf{x} - \mathbf{x}(K)\|_2^2)$$

0. Look at the data



Say you want to classify the grey point into a class. Here, there are three potential classes - lime green, green and orange.

1. Calculate distances



Start by calculating the distances between the grey point and all other points.

2. Find neighbours

Point Distance			
	→ 	2.1	→ 1st NN
	→ 	2.4	→ 2nd NN
	→ 	3.1	→ 3rd NN
	→ 	4.5	→ 4th NN

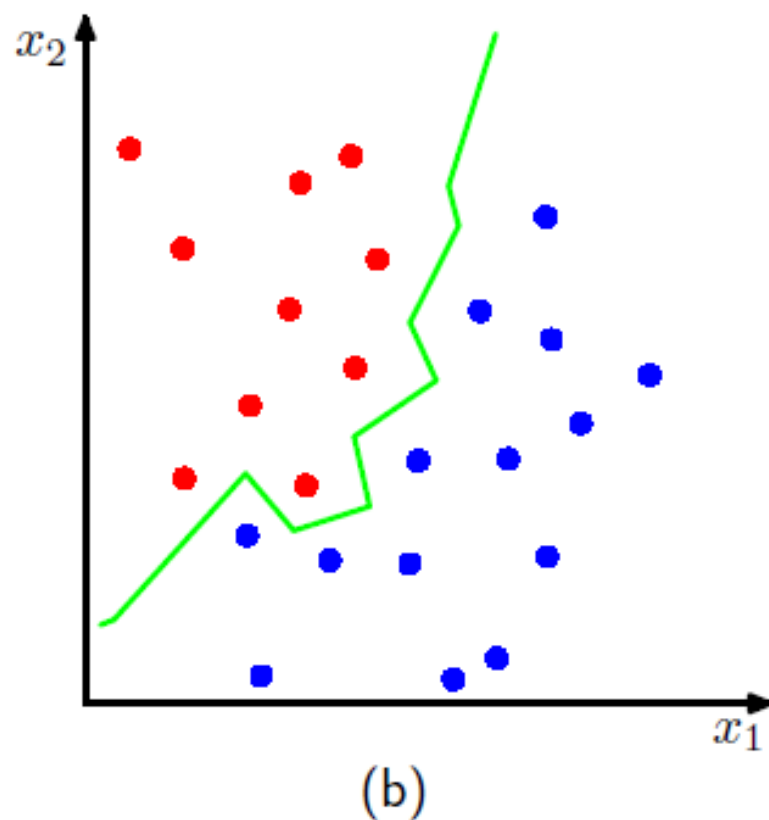
Next, find the nearest neighbours by ranking points by increasing distance. The nearest neighbours (NNs) of the grey point are the ones closest in dataspace.

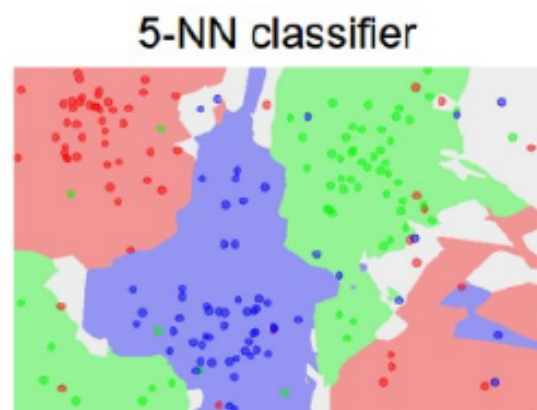
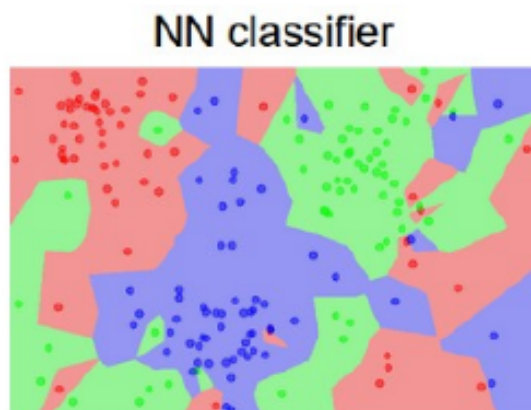
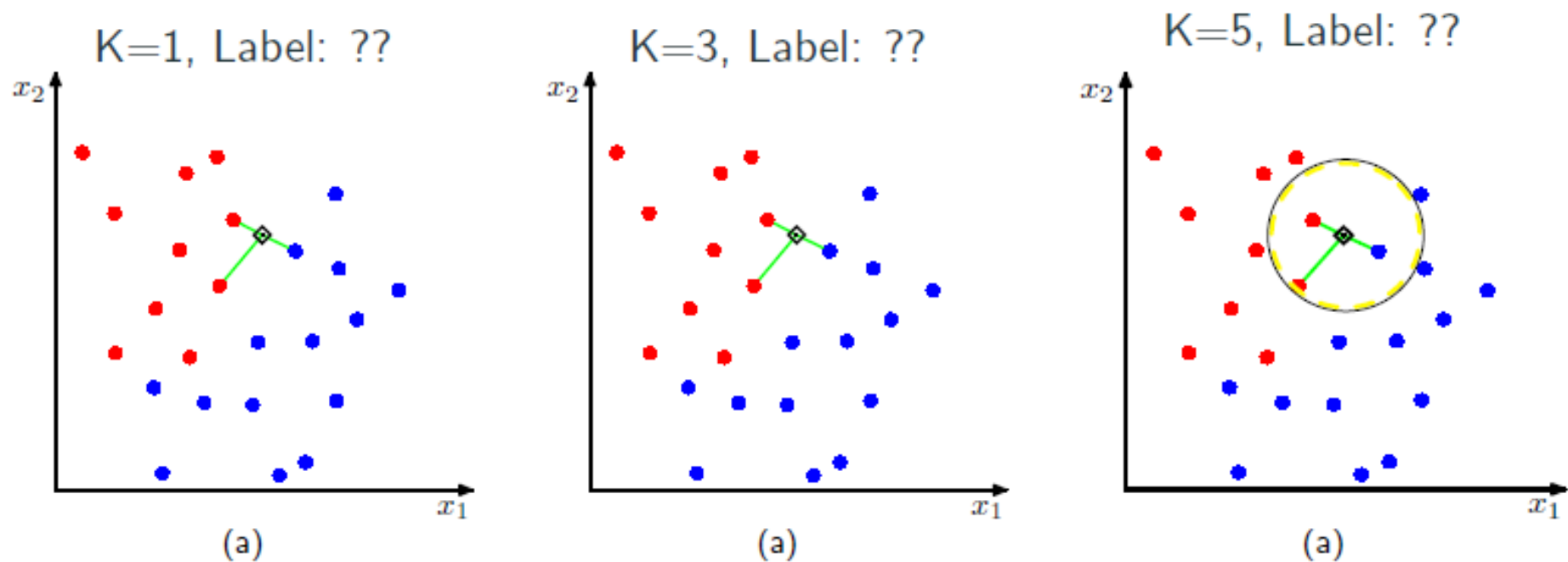
3. Vote on labels

Class	# of votes	
	2	➔ Class  wins the vote! Point  is therefore predicted to be of class  .
	1	
	1	

Vote on the predicted class labels based on the classes of the k nearest neighbours. Here, the labels were predicted based on the k=3 nearest neighbours.

For every point in the space, we can determine its label using the NNC rule. This gives rise to a *decision boundary* that partitions the space into different regions.





Nearest Neighbor Classifier

Memorize training data

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

Nearest Neighbor Classifier

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

For each test image:
Find nearest training image
Return label of nearest image

Advantages of NNC

- Computationally, simple and easy to implement – just compute distances, no optimization required
- Can learn complex decision boundaries

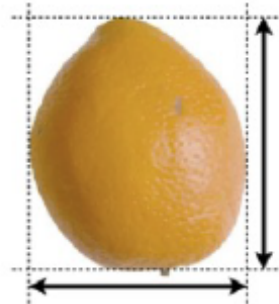
Disadvantages of NNC

- Computationally intensive for large-scale problems:
 - |
- We need to “carry” the training data around. Without it, we cannot do classification. This type of method is called *nonparametric*.
- Choosing the right distance measure and K can be difficult.

Decision Trees and Ensembles

- KNN: Good method with reasonable theoretical guarantees, but not very **explainable**.
- **Decision Trees**
 - ▶ Simple but powerful learning algorithm
 - ▶ More explainable; somehow similar to how people make decisions
 - ▶ One of the most widely used learning algorithms in Kaggle competitions
 - ▶ Lets us introduce **ensembles**, a key idea in ML
- Useful **Information Theoretic** concepts (entropy, mutual information, etc.)

Test example



width > 6.5cm?

Yes

No

height > 9.5cm?

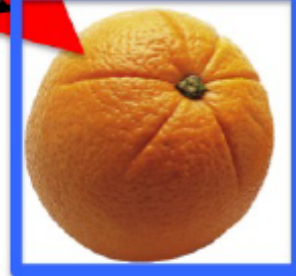
height > 6.0cm?

Yes

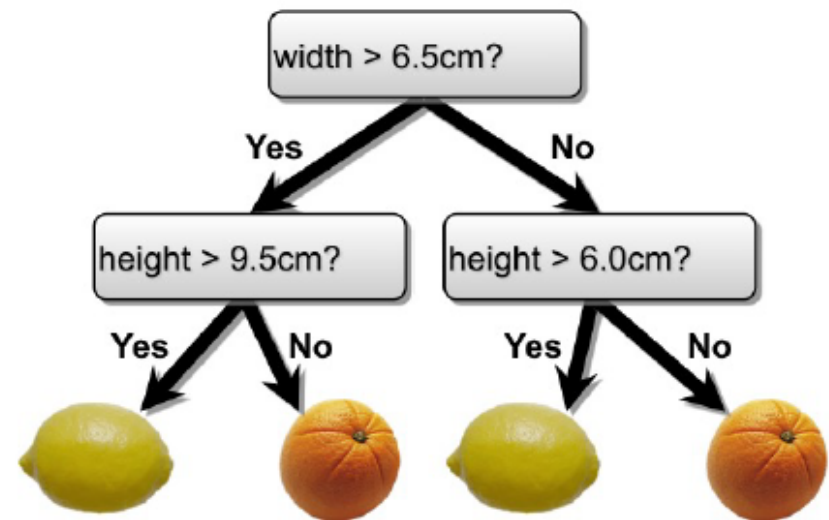
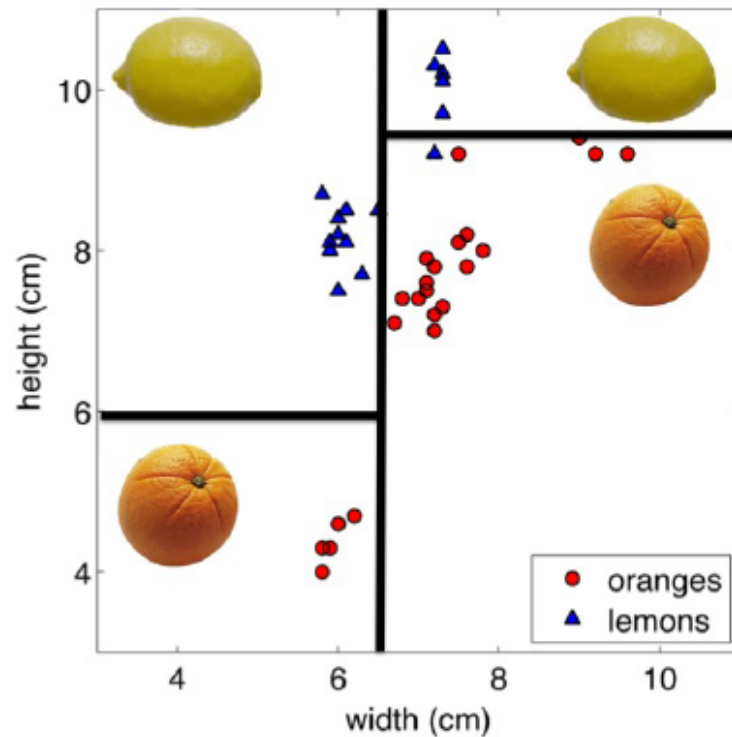
No

Yes

No



- Decision trees make predictions by recursively splitting on different attributes according to a tree structure.



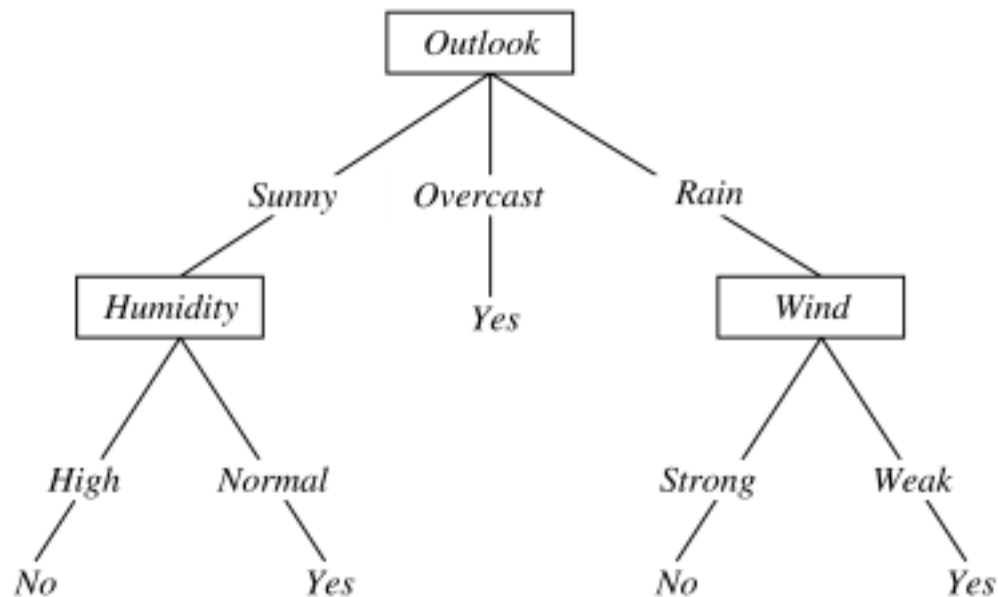
Sample Dataset (was Tennis Played?)

- Columns denote features X_i
- Rows denote labeled instances $\langle x_i, y_i \rangle$
- Class label denotes whether a tennis game was played

$\langle x_i, y_i \rangle$

Predictors				Response
Outlook	Temperature	Humidity	Wind	Class
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

A Decision Tree for *PlayTennis*



Each internal node: **test one feature X_j**

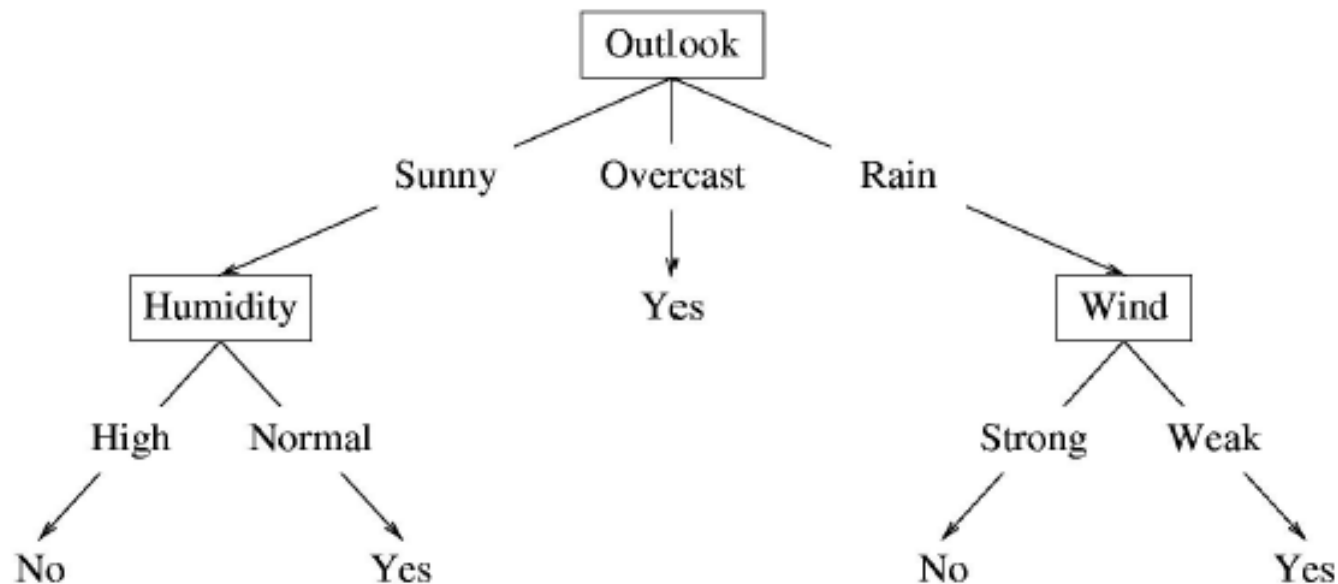
Each branch from a node: **select one value for X_j**

Each leaf node node: **predict Y**

or **$P(Y \mid X \in \text{leaf})$**

Decision Tree

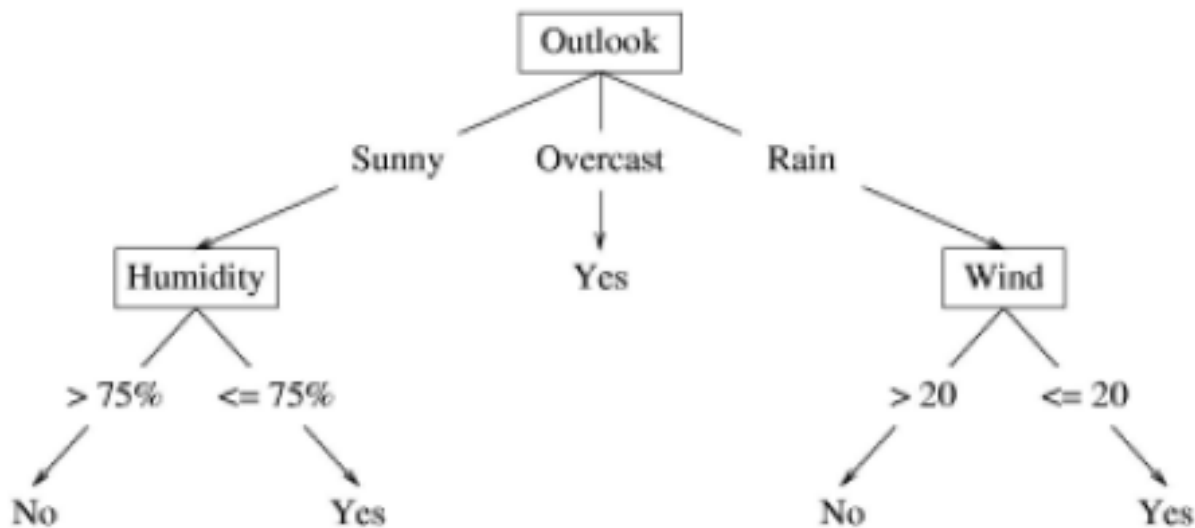
- A possible decision tree for the data:



- What prediction would we make for
<outlook=sunny, temperature=hot, humidity=high, wind=weak> ?

Decision Tree

- If features are continuous, internal nodes can test the value of a feature against a threshold



Example

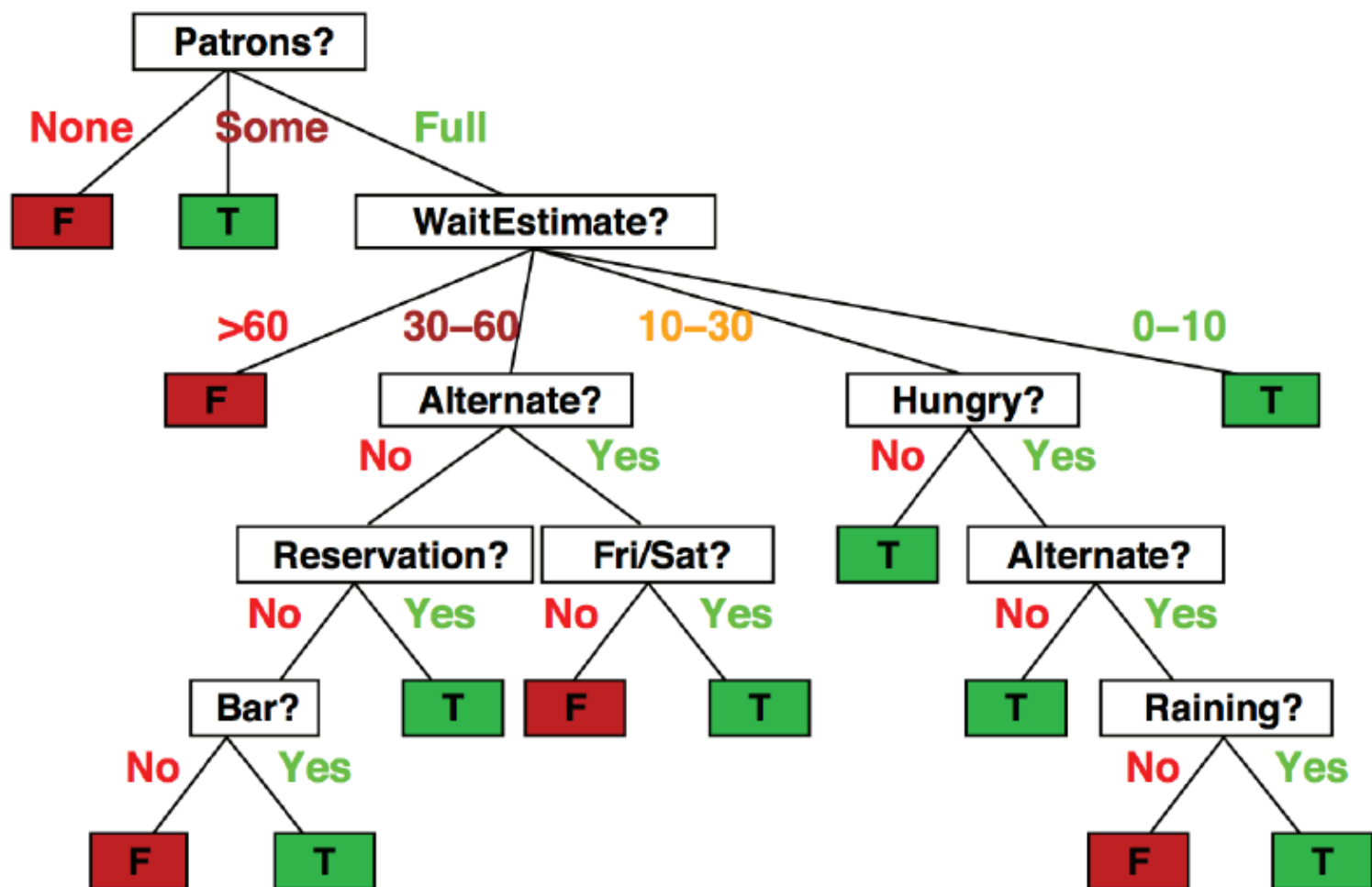
- What if the attributes are discrete?

Example	Input Attributes										Goal
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>	<i>WillWait</i>
x_1	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	$y_1 = \text{Yes}$
x_2	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	$y_2 = \text{No}$
x_3	No	Yes	No	No	Some	\$	No	No	Burger	0-10	$y_3 = \text{Yes}$
x_4	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10-30	$y_4 = \text{Yes}$
x_5	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	$y_5 = \text{No}$
x_6	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	$y_6 = \text{Yes}$
x_7	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	$y_7 = \text{No}$
x_8	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	$y_8 = \text{Yes}$
x_9	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	$y_9 = \text{No}$
x_{10}	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	$y_{10} = \text{No}$
x_{11}	No	No	No	No	None	\$	No	No	Thai	0-10	$y_{11} = \text{No}$
x_{12}	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	$y_{12} = \text{Yes}$

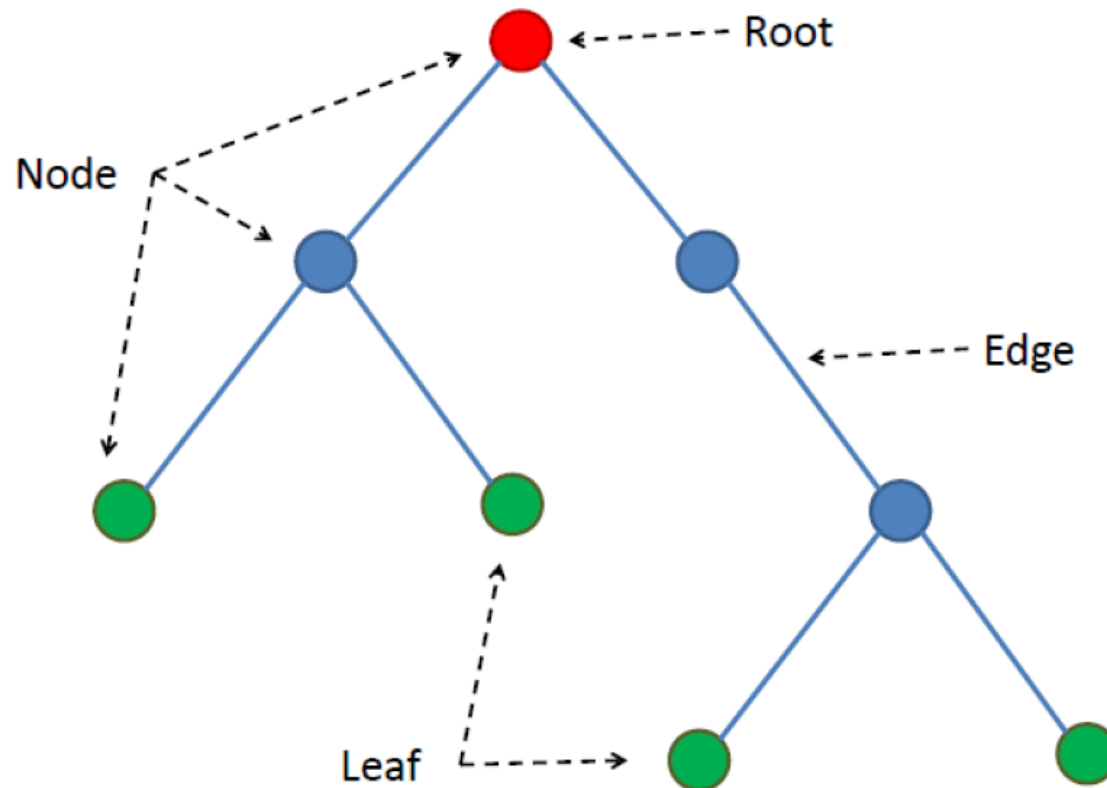
1.	Alternate: whether there is a suitable alternative restaurant nearby.
2.	Bar: whether the restaurant has a comfortable bar area to wait in.
3.	Fri/Sat: true on Fridays and Saturdays.
4.	Hungry: whether we are hungry.
5.	Patrons: how many people are in the restaurant (values are None, Some, and Full).
6.	Price: the restaurant's price range (\$, \$\$, \$\$\$).
7.	Raining: whether it is raining outside.
8.	Reservation: whether we made a reservation.
9.	Type: the kind of restaurant (French, Italian, Thai or Burger).
10.	WaitEstimate: the wait estimated by the host (0-10 minutes, 10-30, 30-60, >60).

Attributes:

- The tree to decide whether to wait (T) or not (F)



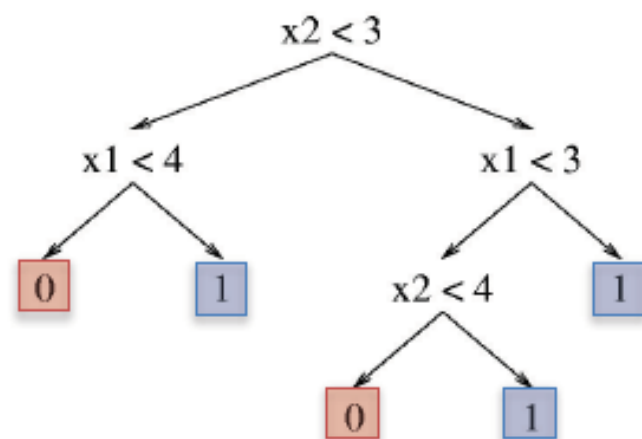
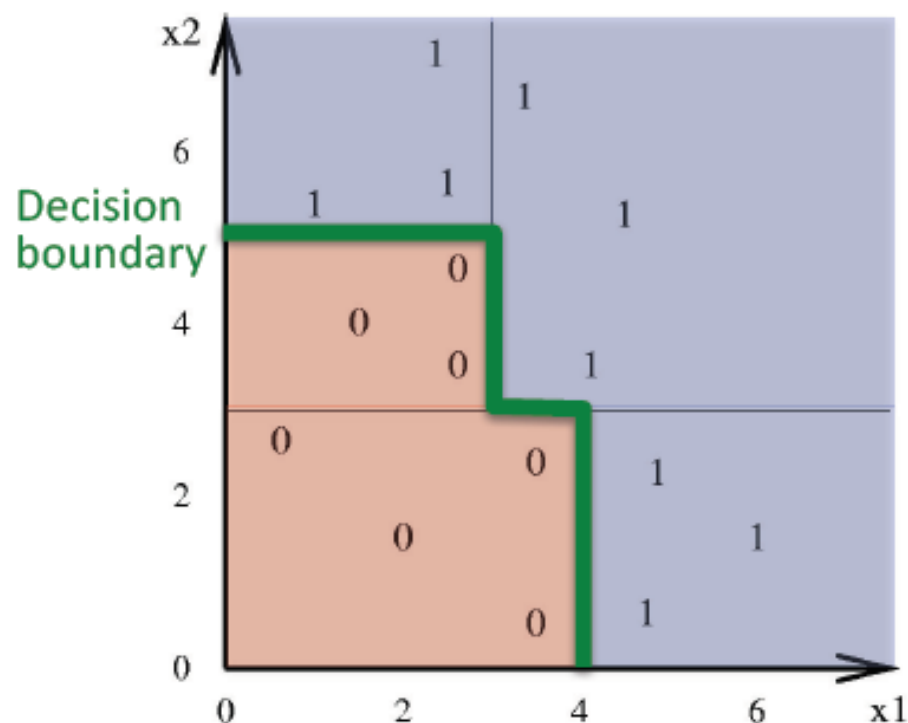
Special names for nodes in a tree Node



- Internal nodes test attributes
- Branching is determined by attribute value
- Leaf nodes are outputs (predictions)

Decision Tree – Decision Boundary

- Decision trees divide the feature space into axis-parallel (hyper-)rectangles
- Each rectangular region is labeled with one label
 - or a probability distribution over labels



How do we learn decision trees?

We want the *smallest* tree that is **consistent with the training data**

(i.e. that assigns the correct labels to training items)

But we **can't enumerate** all possible trees.

$|\mathcal{H}|$ is exponential in the number of features

- In Restaurant example can start from any attribute.
- Any Combination can be used so number of Hypothesis are many/

We use a **heuristic**: greedy top-down search

This is guaranteed to find a consistent tree,
and is biased towards finding smaller trees

Basic Algorithm for Top-Down Learning of Decision Trees

[ID3, C4.5 by Quinlan]

node = root of decision tree

Main loop:

1. $A \leftarrow$ the “best” decision attribute for the next node.
2. Assign A as decision attribute for *node*.
3. For each value of A , create a new descendant of *node*.
4. Sort training examples to leaf nodes.
5. If training examples are perfectly classified, stop. Else, recurse over new leaf nodes.

How do we choose which attribute is best?

Choosing the Best Attribute

Key problem: choosing which attribute to split a given set of examples

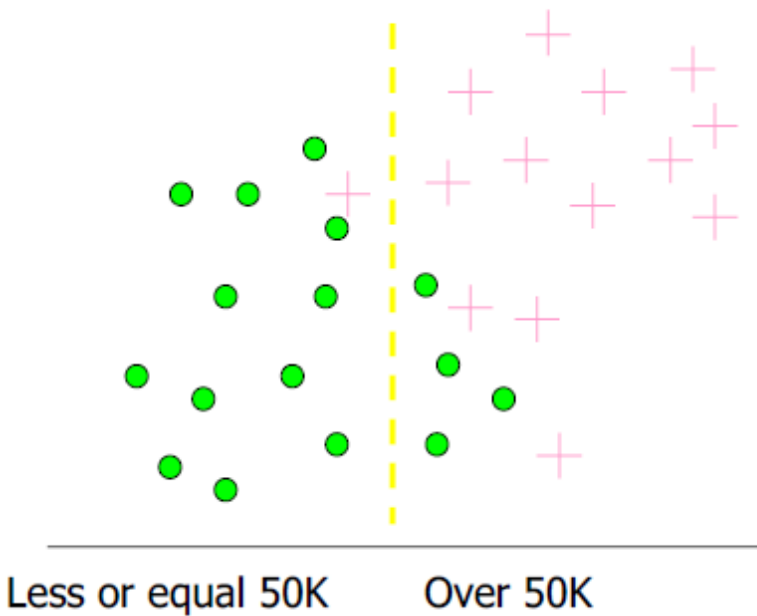
- Some possibilities are:
 - **Random:** Select any attribute at random
 - **Least-Values:** Choose the attribute with the smallest number of possible values
 - **Most-Values:** Choose the attribute with the largest number of possible values
 - **Max-Gain:** Choose the attribute that has the largest expected *information gain*
- The ID3 algorithm uses the Max-Gain method of selecting the best attribute

Information Gain

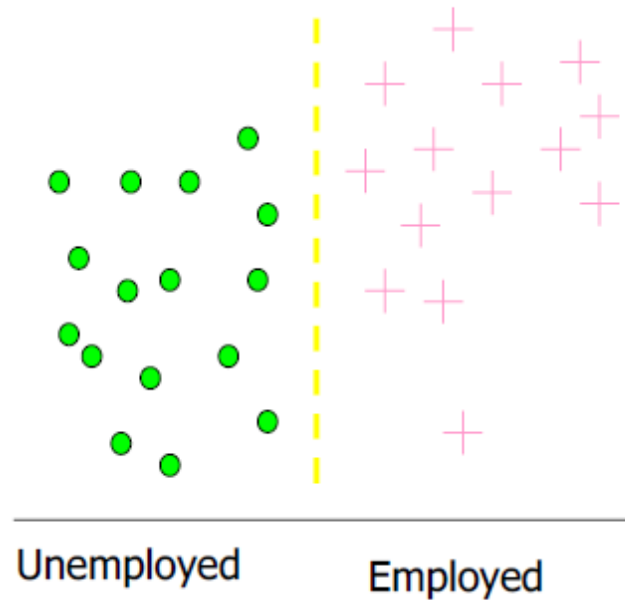
Which test is more informative?

Loan
Example

**Split over whether
Balance exceeds 50K**



**Split over whether
applicant is employed**



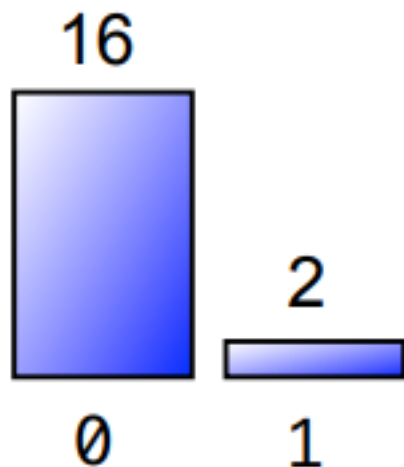
Q: Which coin is more uncertain?

Sequence 1:

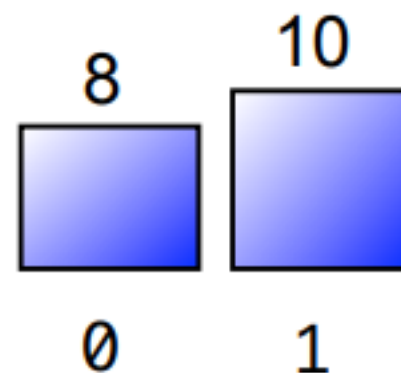
0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 ... ?

Sequence 2:

0 1 0 1 0 1 1 1 0 1 0 0 1 1 0 1 0 1 ... ?



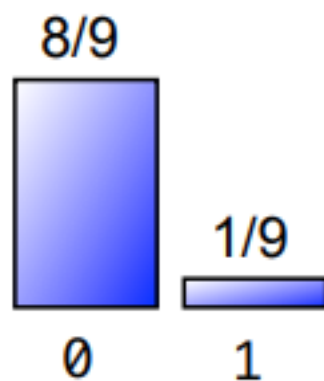
versus



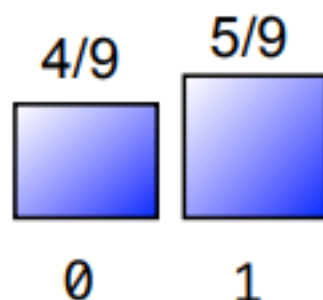
Uncertainty = How hard it is to predict the next outcome.

Entropy is a measure of expected “surprise”: How uncertain are we of the value of a draw from this distribution?

$$H(X) = -\mathbb{E}_{X \sim p}[\log_2 p(X)] = -\sum_{x \in X} p(x) \log_2 p(x)$$



$$-\frac{8}{9} \log_2 \frac{8}{9} - \frac{1}{9} \log_2 \frac{1}{9} \approx \frac{1}{2}$$

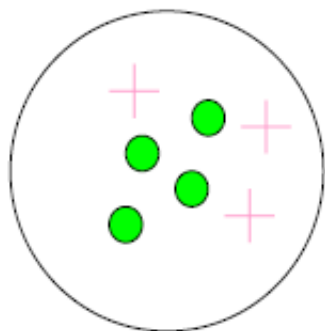


$$-\frac{4}{9} \log_2 \frac{4}{9} - \frac{5}{9} \log_2 \frac{5}{9} \approx 0.99$$

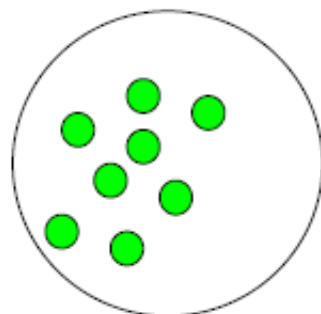
Information Gain

Impurity/Entropy (informal)

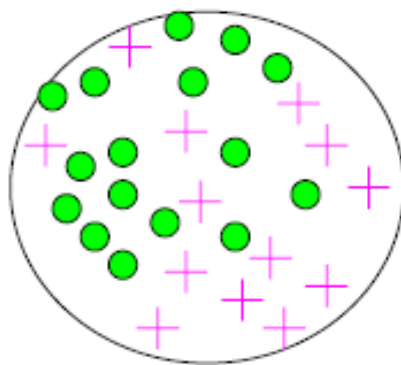
- Measures the level of **impurity** in a group of examples



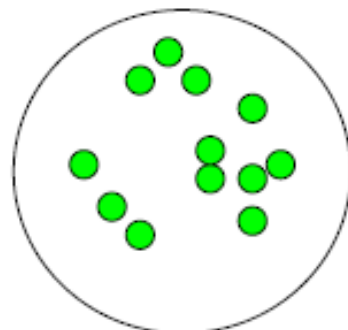
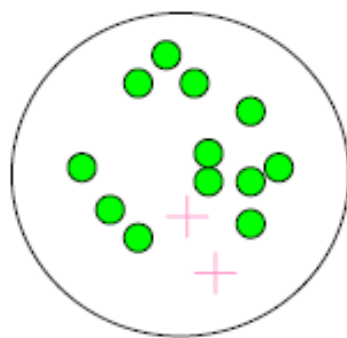
Very impure group



Less impure



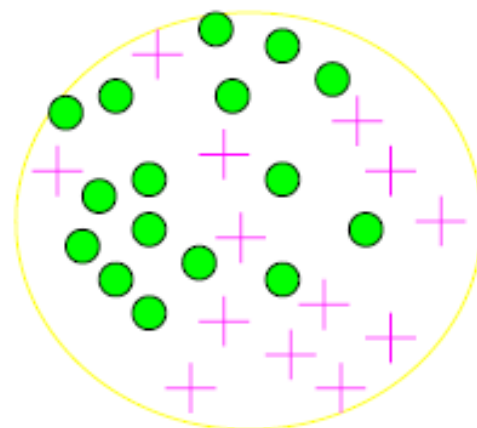
**Minimum
impurity**



Entropy: a common way to measure impurity

- Entropy =
$$\sum_i -p_i \log_2 p_i$$

p_i is the probability of class i



- Entropy comes from information theory. The higher the entropy the more the information content.

What does that mean for learning from examples?

2-Class Cases:

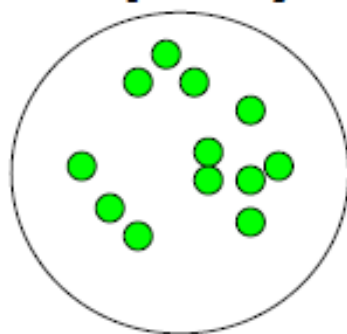
$$\text{Entropy } H(x) = - \sum_{i=1}^n P(x = i) \log_2 P(x = i)$$

- What is the entropy of a group in which all examples belong to the same class?

– entropy = $-1 \log_2 1 = 0$

not a good training set for learning

**Minimum
impurity**

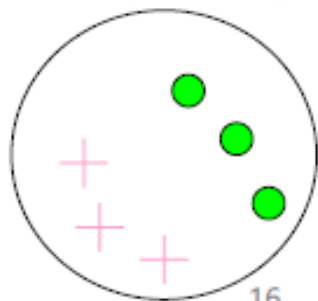


- What is the entropy of a group with 50% in either class?

– entropy = $-0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$

good training set for learning

**Maximum
impurity**



Information Gain

- We want to determine **which attribute** in a given set of training feature vectors is **most useful** for discriminating between the classes to be learned.
- **Information gain** tells us how important a given attribute of the feature vectors is.
- We will use it to decide the ordering of attributes in the nodes of a decision tree.

From Entropy to Information Gain

Entropy $H(X)$ of a random variable X

$$H(X) = - \sum_{i=1}^n P(X = i) \log_2 P(X = i)$$

Specific conditional entropy $H(X|Y=v)$ of X given $Y=v$:

$$H(X|Y = v) = - \sum_{i=1}^n P(X = i|Y = v) \log_2 P(X = i|Y = v)$$

Conditional entropy $H(X|Y)$ of X given Y :

$$H(X|Y) = \sum_{v \in \text{values}(Y)} P(Y = v) H(X|Y = v)$$

Mutual information (aka Information Gain) of X and Y :

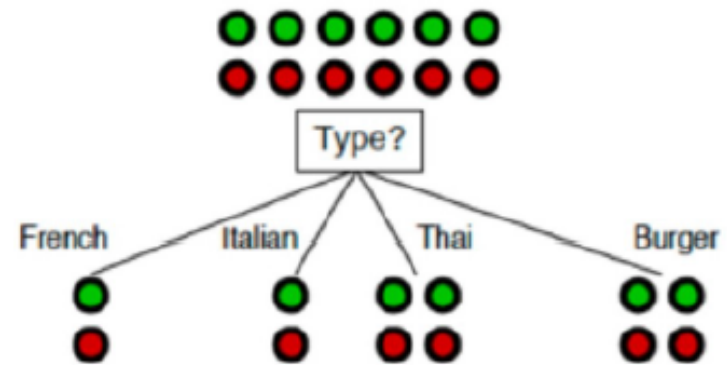
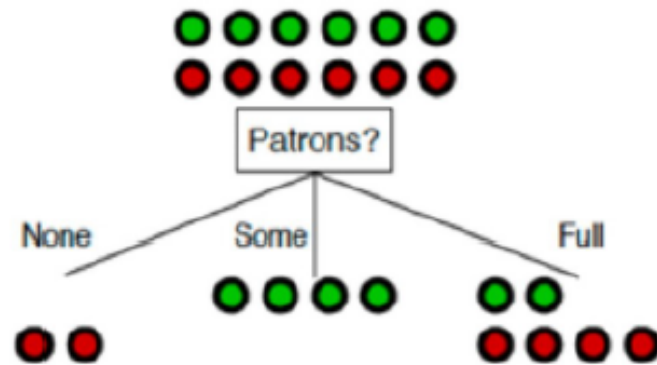
$$I(X, Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

Restaurant Example to find Information Gain

Attributes										Target
<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>	<i>WillWait</i>
<i>T</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>Some</i>	<i>\$\$\$</i>	<i>F</i>	<i>T</i>	<i>French</i>	<i>0-10</i>	<i>T</i>
<i>T</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>Full</i>	<i>\$</i>	<i>F</i>	<i>F</i>	<i>Thai</i>	<i>30-60</i>	<i>F</i>
<i>F</i>	<i>T</i>	<i>F</i>	<i>F</i>	<i>Some</i>	<i>\$</i>	<i>F</i>	<i>F</i>	<i>Burger</i>	<i>0-10</i>	<i>T</i>
<i>T</i>	<i>F</i>	<i>T</i>	<i>T</i>	<i>Full</i>	<i>\$</i>	<i>F</i>	<i>F</i>	<i>Thai</i>	<i>10-30</i>	<i>T</i>
<i>T</i>	<i>F</i>	<i>T</i>	<i>F</i>	<i>Full</i>	<i>\$\$\$</i>	<i>F</i>	<i>T</i>	<i>French</i>	<i>>60</i>	<i>F</i>
<i>F</i>	<i>T</i>	<i>F</i>	<i>T</i>	<i>Some</i>	<i>\$\$</i>	<i>T</i>	<i>T</i>	<i>Italian</i>	<i>0-10</i>	<i>T</i>
<i>F</i>	<i>T</i>	<i>F</i>	<i>F</i>	<i>None</i>	<i>\$</i>	<i>T</i>	<i>F</i>	<i>Burger</i>	<i>0-10</i>	<i>F</i>
<i>F</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>Some</i>	<i>\$\$</i>	<i>T</i>	<i>T</i>	<i>Thai</i>	<i>0-10</i>	<i>T</i>
<i>F</i>	<i>T</i>	<i>T</i>	<i>F</i>	<i>Full</i>	<i>\$</i>	<i>T</i>	<i>F</i>	<i>Burger</i>	<i>>60</i>	<i>F</i>
<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>Full</i>	<i>\$\$\$</i>	<i>F</i>	<i>T</i>	<i>Italian</i>	<i>10-30</i>	<i>F</i>
<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>None</i>	<i>\$</i>	<i>F</i>	<i>F</i>	<i>Thai</i>	<i>0-10</i>	<i>F</i>
<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>Full</i>	<i>\$</i>	<i>F</i>	<i>F</i>	<i>Burger</i>	<i>30-60</i>	<i>T</i>

Use the attributes to decide whether to wait (T) or not wait (F)

Restaurant Example to find Information Gain



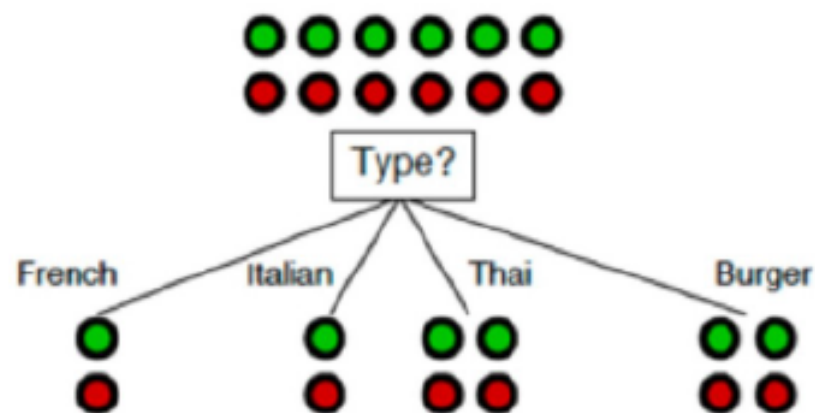
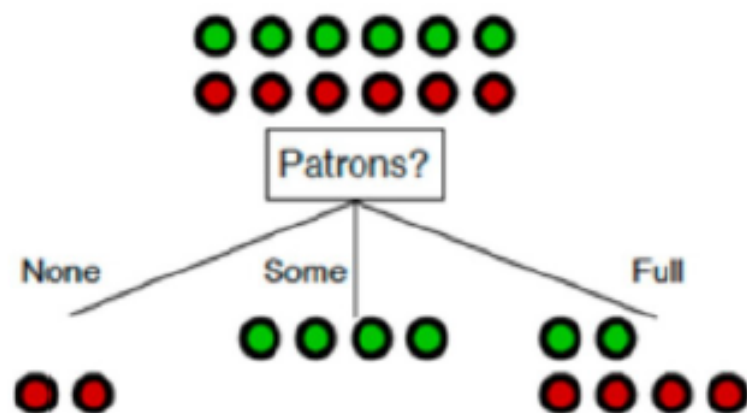
Patron vs. Type?

- Let us compute the information gain $I(X; Y) = H[Y] - H[Y|X]$ for Patron and Type
- $H(Y) = -\frac{6}{12} \log \frac{6}{12} - \frac{6}{12} \log \frac{6}{12} = 1$ bit
- $H(Y|X = \text{none}) = 0$
- $H(Y|X = \text{some}) = 0$
- $H(Y|X = \text{full}) = -\left(\frac{2}{2+4} \log \frac{2}{2+4} + \frac{4}{2+4} \log \frac{4}{2+4}\right) \approx 0.9$ bits
- Thus the conditional entropy is

$$H(Y|X) = \left(\frac{2}{12} \times 0 + \frac{4}{12} \times 0 + \frac{6}{12} \times 0.9\right) = 0.45 \text{ bits}$$

- Information Gain $I(X; Y) = 1 - 0.45 = 0.55$ bits

Restaurant Example to find Information Gain



- $H(Y) = -\frac{6}{12} \log \frac{6}{12} - \frac{6}{12} \log \frac{6}{12} = 1$ bit

- $H(Y|X = \text{french}) = \log 2 = 1$ bit

- $H(Y|X = \text{italian}) = \log 2 = 1$ bit

- $H(Y|X = \text{thai}) = \log 2 = 1$ bit

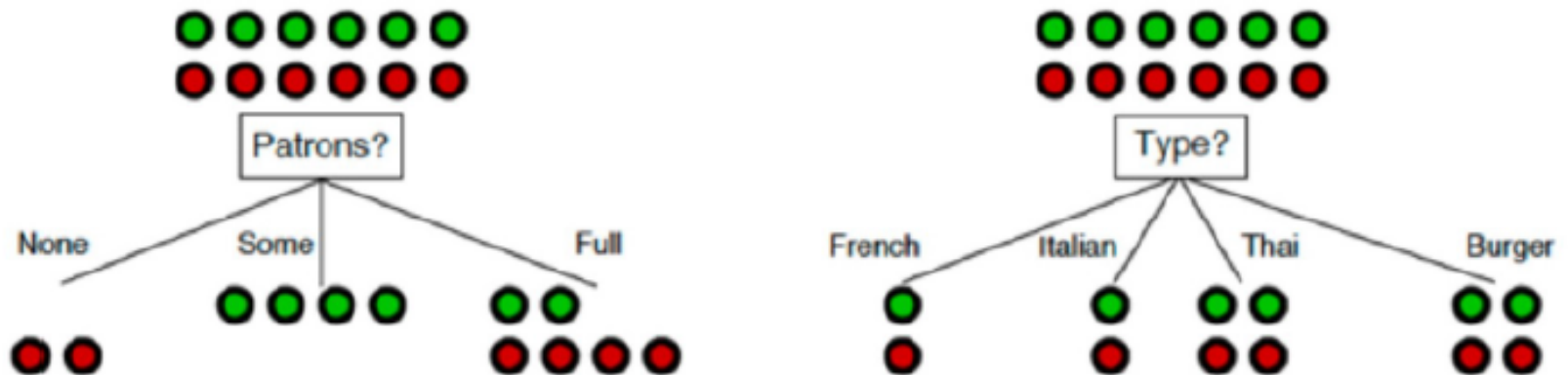
- $H(Y|X = \text{burger}) = \log 2 = 1$ bit

- Thus the conditional entropy is

$$H(Y|X) = \frac{2}{12} \times 1 + \frac{2}{12} \times 1 + \frac{4}{12} \times 1 + \frac{4}{12} \times 1 = 1 \text{ bit}$$

- Information Gain $I(X; Y) = 1 - 1 = 0$ bits

Restaurant Example to find Information Gain



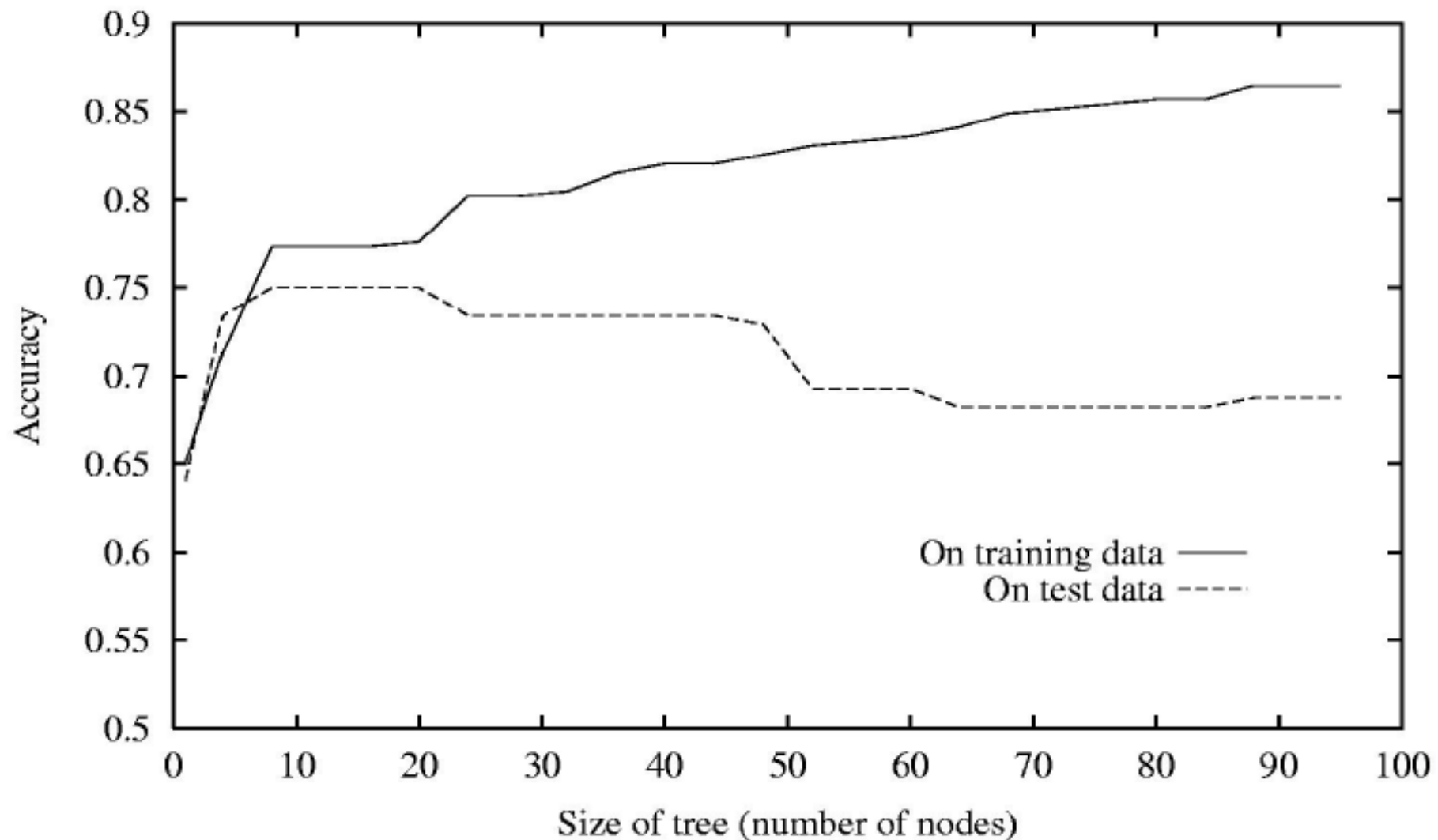
- Information gain from “Patron” is 0.55 bits.
- Information gain from “Type” is 0 bits.

Thus, we should split on “Patron” and not “Type” (higher information gain). This is consistent with our intuition.

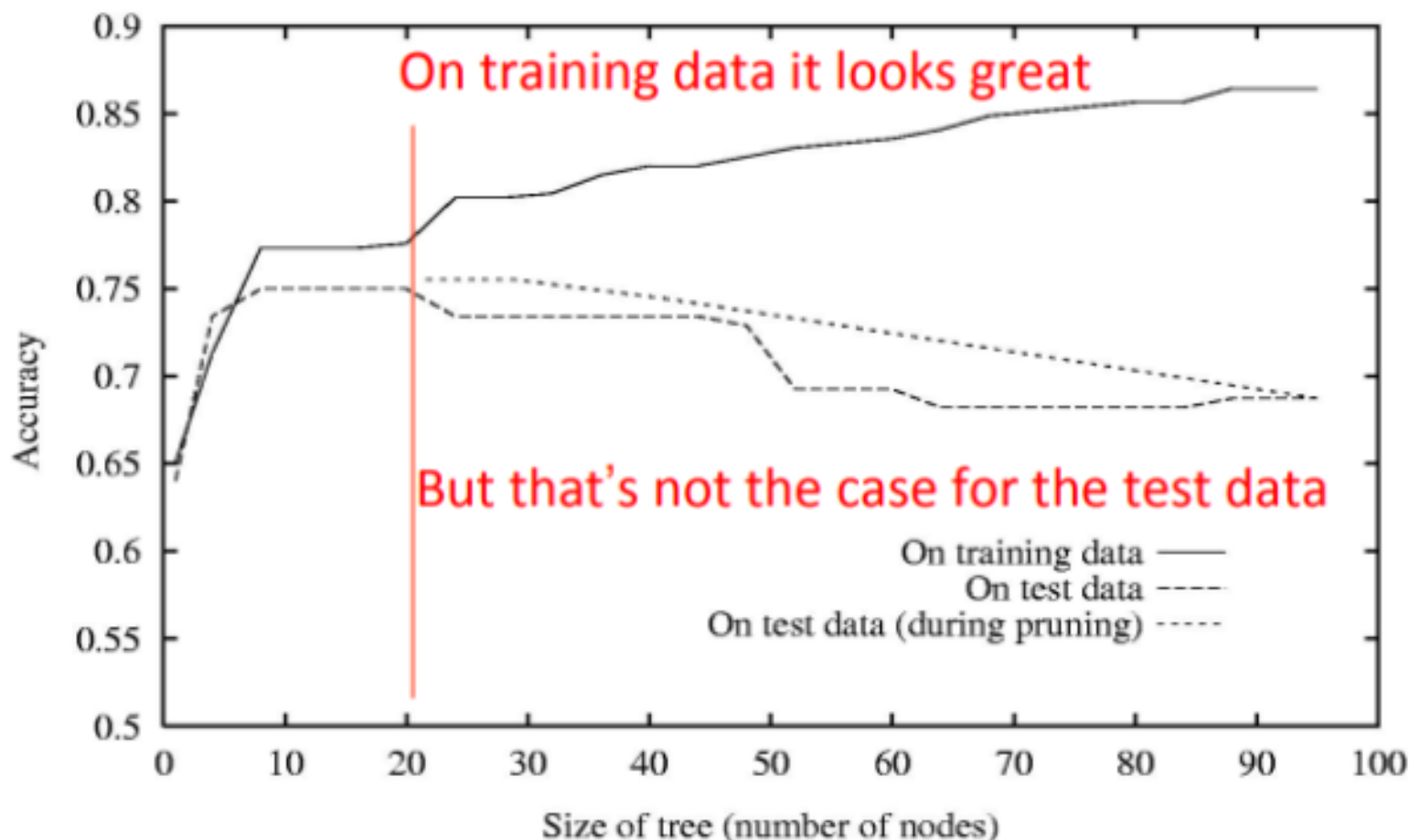
Overfitting in Decision Trees

- Irrelevant attributes can result in *overfitting* the training example data
 - If hypothesis space has many dimensions (large number of attributes), we may find **meaningless regularity** in the data that is irrelevant to the true, important, distinguishing features
- If we have too little training data, even a reasonable hypothesis space will ‘overfit’

Overfitting in Decision Trees



Effect of Reduced-Error Pruning



The tree is pruned back to the red line where it gives more accurate results on the test data

Limitations of Decision Tree Models

Decision trees models are highly interpretable and fast to train, using our greedy learning algorithm.

However, in order to capture a complex decision boundary (or to approximate a complex function), we need to use a large tree (since each time we can only make axis aligned splits).

We've seen that large trees have high variance and are prone to overfitting.

For these reasons, in practice, decision tree models often underperforms when compared with other classification or regression methods.

Bagging

One way to adjust for the high variance of the output of an experiment is to perform the experiment multiple times and then average the results.

The same idea can be applied to high variance models:

1. **(Bootstrap)** we generate multiple samples of training data, via bootstrapping. We train a full decision tree on each sample of data.
2. **(Aggregate)** for a given input, we output the averaged outputs of all the models for that input.

For classification, we return the class that is outputted by the plurality of the models.

This method is called **Bagging** (Breiman, 1996), short for, of course, Bootstrap Aggregating.

Bagging

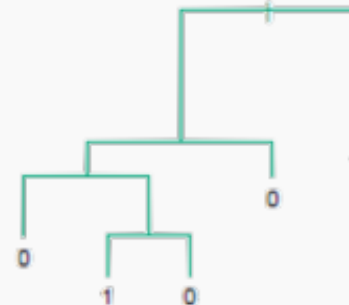
Original Data

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
X_4	y_4
X_5	y_5
$\vdots$	$\vdots$
X_n	y_n

Bootstrap Sample 1

X	Y
X_4	y_4
X_{14}	y_{14}
X_{11}	y_{11}
X_2	y_2
X_{35}	y_{35}
$\vdots$	$\vdots$
X_k	y_k

Decision Tree 1



Used and unused data

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
X_4	y_4
X_5	y_5
$\vdots$	$\vdots$
X_n	y_n

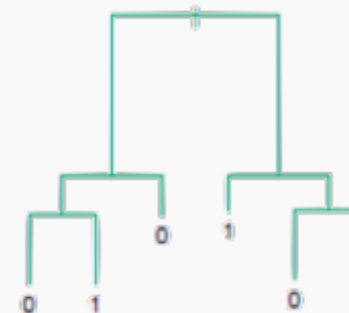
Original Data

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
X_4	y_4
X_5	y_5
$\vdots$	$\vdots$
X_n	y_n

Bootstrap Sample 2

X	Y
X_5	y_5
X_3	y_3
X_{12}	y_{12}
X_{43}	y_{43}
X_1	y_1
$\vdots$	$\vdots$
X_k	y_k

Decision Tree 2



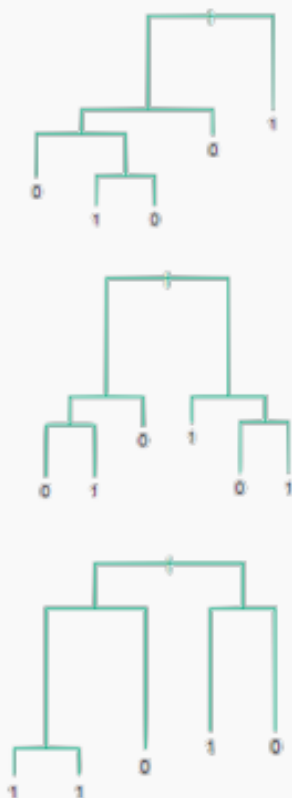
Used and unused data

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
X_4	y_4
X_5	y_5
$\vdots$	$\vdots$
X_n	y_n

Prediction

B Trees that did not see $\{X_i, y_i\}$

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
$\vdots$	$\vdots$
X_i	y_i
$\vdots$	$\vdots$
X_n	y_n



Classification

$$\hat{y}_{i,pw} = \text{majority}(\hat{y}_i)$$

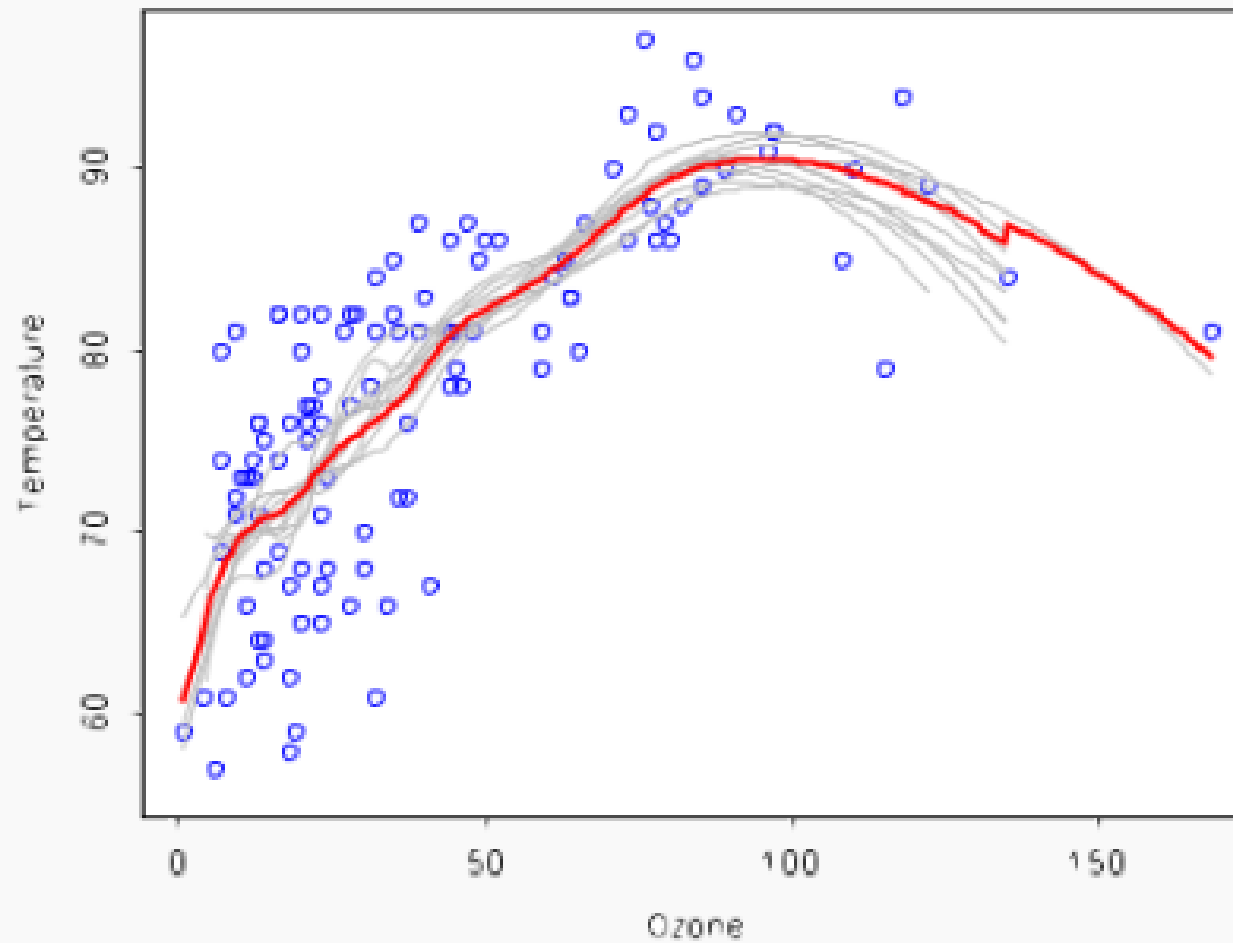
$$e_i = \mathbb{I}(\hat{y}_{i,pw} \neq y_i)$$

Regression

$$\hat{y}_{i,pw} = \sum_{j \in B} \hat{y}_{i,j}$$

$$e_i = (y_i - \hat{y}_{i,pw})^2$$

Bagging



Bagging

Question: Do you see any problems?

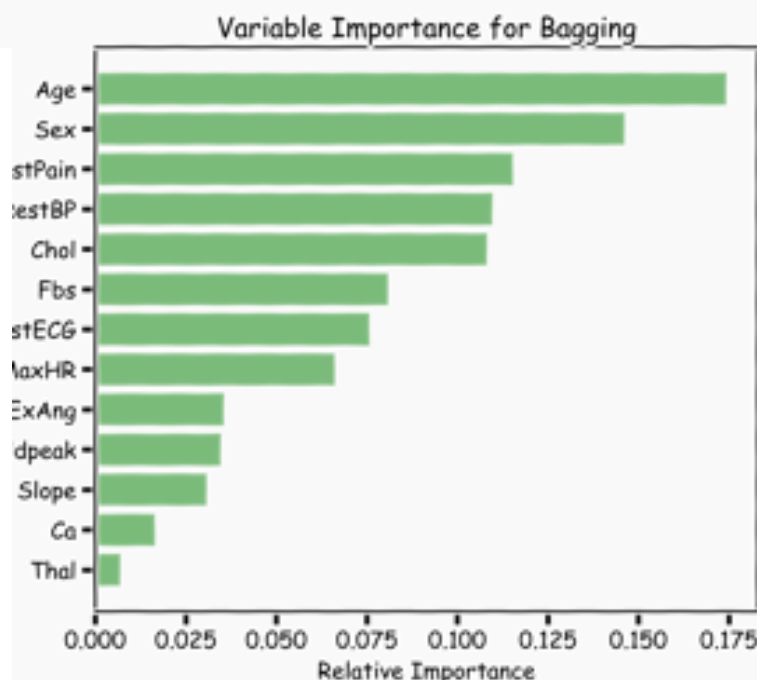
- Still some overfitting if the trees are too large.
- If trees are too shallow it can still underfits.
- Interpretability:

The **major drawback** of bagging (and other **ensemble methods** that we will study) is that the averaged model is no longer easily interpretable - i.e. one can no longer trace the 'logic' of an output through a series of decisions based on predictor values!

Variable Importance for Bagging

Bagging improves prediction accuracy at the expense of interpretability.

Calculate the total amount that the MSE (for regression) or Gini index (for classification) is decreased due to splits over a given Feature, averaged over all B trees.



Improving on Bagging

In practice, the ensembles of trees in Bagging tend to be highly correlated.

Suppose we have an extremely strong Feature, x_j , in the training set amongst moderate Features. Then the greedy learning algorithm ensures that most of the models in the ensemble will choose to split on x_j in early iterations.

Each tree in the ensemble is identically distributed, with the expected output of the averaged model the same as the expected output of any of the trees.

Random Forests

Random Forest is a modified form of bagging that creates ensembles of independent decision trees.

To de-correlate the trees, we:

1. train each tree on a separate bootstrap sample of the full training set (same as in bagging)
2. for each tree, at each split, we **randomly select a set of J' Features from the full set of Features.**

From amongst the J' Features, we select the optimal Feature and the optimal corresponding threshold for the split.

Scenario

- Task: Predict whether a person will buy a car (Yes/No) based on features like income, age, and location.

Income (\$)	Age	Location	Buy Car (Yes/No)
50,000	25	Urban	No
60,000	35	Suburban	Yes
45,000	23	Rural	No
80,000	45	Suburban	Yes
70,000	40	Urban	Yes

Bagging

1. Create multiple bootstrapped datasets by sampling with replacement.
 - Example:
 - Dataset 1: Row 1, Row 3, Row 5, Row 1, Row 4
 - Dataset 2: Row 2, Row 4, Row 3, Row 5, Row 2
 2. Train a separate decision tree on each dataset.
 3. Use majority vote for classification or average prediction for regression.
- Example Outcome:
 - Tree 1 predicts: No, No, Yes, Yes, Yes → Final Vote: **Yes**
 - Tree 2 predicts: Yes, Yes, Yes, No, No → Final Vote: **Yes**
 - Aggregated Result: Majority vote → **Yes (Buys Car)**

Random Forest

1. Create bootstrapped datasets (same as Bagging).
2. For each decision tree:
 - At every split, select a random subset of features.
 - Example:
 - Tree 1: Split on Income and Age
 - Tree 2: Split on Income and Location
 - Tree 3: Split on Age and Location
3. Use majority vote for classification or average prediction for regression.

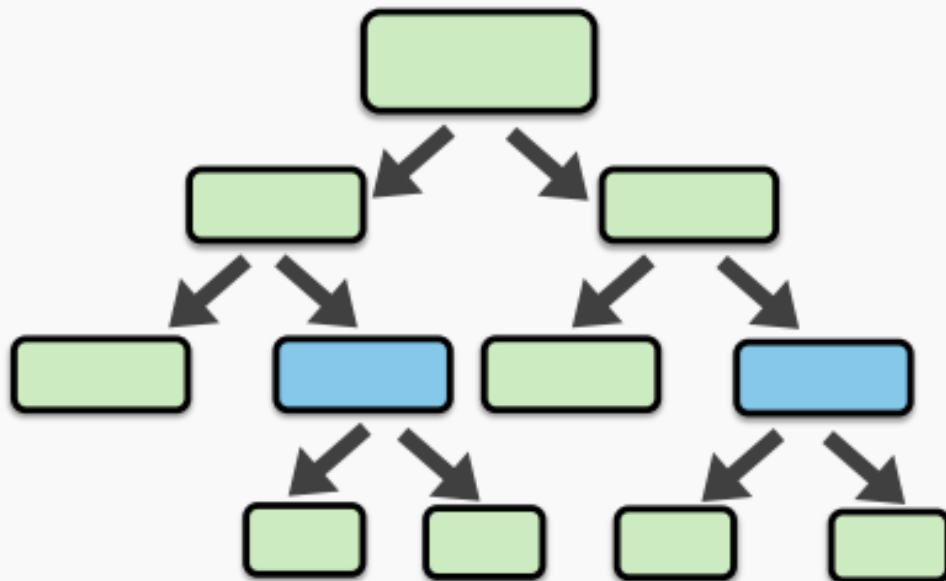
Example Outcome:

- Tree 1 uses Income and Age → Predicts **Yes**
- Tree 2 uses Income and Location → Predicts **No**
- Tree 3 uses Age and Location → Predicts **Yes**
- Aggregated Result: Majority vote → **Yes (Buys Car)**

Random Forest reduces both **variance** and **overfitting** by decorrelating trees through feature randomness.

DECISION TREE ISSUES?

- Shallow trees:
 - Shallow trees (with very few leaves) suffer from high bias and do not train well.
- Deep Trees:
 - Deep trees (with large number of nodes and leaves) have low bias, but suffer from high variance leading to



RF ISSUES?

- Variance:
 - Although variance reduction is better than bagging, the generalization error is still high
- Speed:
 - Large number of trees can make the algorithm very slow and ineffective for real-time predictions

